AI Pioneers

CCA-F | Module 1

Lab 1.1

Building the Agentic Loop: Orchestration & Subagent Coordination

---

Scenario: Enterprise Customer Support Triage Agent

Module	M1 — Agentic Architecture & Orchestration
Lab type	Mandatory · Instructor-Led
Duration	~60 minutes (hands-on)
Environment	Blue Labs VM · Claude Code CLI · Python 3
Sections	S1 (Agentic Loop) · S2 (Multi-Agent Orchestration)
	S3 (Context Passing) · S4 (Step Enforcement)

Lab Objectives

By the end of this lab you will be able to:

- Build a stable agentic loop that reads `stop_reason` correctly so the agent knows when to act, continue, or halt — e.g. a research assistant that keeps calling tools until the task is truly done.
- Apply the coordinator–subagent (hub-and-spoke) pattern so one orchestrator delegates to specialists — e.g. a lead agent routing "summarize," "translate," and "validate" to dedicated subagents.
- Pass explicit context into subagents and enforce multi-step order programmatically — e.g. a document pipeline where extraction must finish before analysis begins.

1. Overview

This lab teaches you how to wire an agentic system that handles real work — not a toy prompt-and-reply loop, but a pipeline that calls tools, inspects results, delegates to specialist agents, and enforces processing order in code. You will build this around a customer support triage scenario that any enterprise deployment team will recognise.

SN	Section	What you will build	Core concept
Ex 1	**S1 — Agentic Loop**	A loop that calls tools until a support ticket is fully classified — not before, not after.	stop_reason drives the loop; tool results are appended back into messages.
Ex 2	**S2 — Orchestration**	A coordinator that reads a new ticket and delegates to three specialist subagents in sequence.	Hub-and-spoke: subagents have no shared memory; the coordinator owns the state.
Ex 3	**S3 — Context Passing**	A typed TicketContext object that travels through the pipeline and makes every handoff explicit and safe.	Structured context is passed explicitly; implicit assumptions produce wrong answers.
Ex 4	**S4 — Step Enforcement**	Programmatic gates that block downstream steps until upstream steps have produced a verified result.	Code enforces order; prompt instructions alone are unreliable under load.

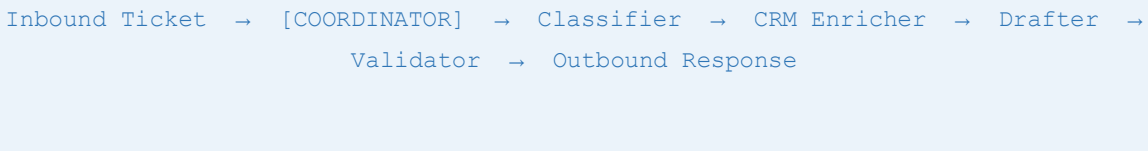
2. Scenario

You are building a support triage agent for Arctive, a mid-size B2B SaaS company. Arctive's support team receives hundreds of tickets per day across email, chat, and web form. Today, every ticket lands in a shared queue and is manually routed to the correct team. That process is slow, inconsistent, and does not scale.

Your task is to build the AI backbone of an automated triage pipeline. The pipeline must:

1. Classify the ticket: product area, severity, and intent (bug / question / feature request / billing).
2. Enrich the ticket: pull the customer's account tier, open contracts, and recent case history from the CRM.
3. Draft a first-response: a contextually accurate reply the agent or human reviewer can send or edit.
4. Validate before dispatch: confirm the draft references the correct product, matches the customer's tier, and meets SLA tone requirements.

Pipeline Flow



Each arrow is an explicit context handoff. Each box is an independent agent call.

Why four subagents?

You could write one giant prompt that classifies, enriches, drafts, and validates in one call. In practice this breaks down: classification errors silently contaminate the draft; CRM data is missing and nobody flags it; the validator has no clear contract to check against. Separating concerns makes each step testable, retryable, and replaceable — the foundations of a production-grade system.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Building with the Claude API	Agentic loop · Tool use · Programmatic gating	**S1, S4**
Introduction to Subagents	What are subagents? · Designing effective subagents · Creating & using subagents	**S2, S3**

The lab assumes you already understand stop_reason, the tool_use loop, and the coordinator–subagent memory boundary. The instructor will not re-teach these — bring your questions to Doubt Session 1 instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

5. Start your Blue Labs VM: open https://www.bluelabs.studio/, locate your VM, click Start, then Connect.
6. Open a terminal in the VM. Verify Claude Code is available:
   ```
   claude --version
   ```
7. Verify Python 3:
   ```
   python3 --version
   ```
8. Install the Anthropic SDK if not present:
   ```
   pip install anthropic
   ```
9. Create and enter your working directory:
   ```
   mkdir -p ~/lab1_1 && cd ~/lab1_1
   ```

ANTHROPIC_API_KEY

Your API key is pre-configured in the Blue Labs VM as an environment variable. You do not need to set it manually. If you receive an authentication error, run: echo $ANTHROPIC_API_KEY to verify it is present, and ask the instructor if it is empty.

4. Exercises

Work through all four exercises in order — each one builds on the previous. Suggested timings are shown per exercise. Prioritise understanding over completion; the extension challenges at the end are for fast finishers.

Exercise 1: The Agentic Loop (S1) ~15 min

Background

An agentic loop is not a for-loop with a fixed iteration count. It runs until Claude says it is done. The mechanism is stop_reason: every response from the API carries a value that tells your code what to do next.

stop_reason	What it means	What your loop does
`"tool_use"`	Claude wants to invoke a tool.	Execute the tool, append the result as a user message, loop again.
`"end_turn"`	Claude has finished its response.	Extract the text from content[], break out of the loop.
`"max_tokens"`	Response was cut at the token limit.	Log a warning; consider summarising history and retrying.
`"stop_sequence"`	A custom stop string was matched.	Treat as end_turn unless you have specific handling logic.

Task

Build a ticket-classifier agent that keeps calling a classification tool until it has all the fields it needs: product_area, severity, and intent. It must not stop early and must not loop after all fields are confirmed.

Step 1 — Create tools.py

Write a Python function called classify_ticket that accepts two arguments: ticket_text (a string) and fields_needed (a list of strings). The function should return a dictionary containing only the requested fields with a value for each. Use the following as your field vocabulary:

- product_area — one of: Billing, Platform, Integrations, Security, Onboarding
- severity — one of: P1-Critical, P2-High, P3-Medium, P4-Low
- intent — one of: Bug, Question, Feature Request, Billing Dispute

For this lab, values can be simulated (e.g. randomly selected from the lists above). In production, this function would call a real classification model or rules engine. Focus on getting the function signature and return shape right.

Step 2 — Create loop.py

Write the agentic loop that drives the classifier. Use the following ticket as your test input throughout all exercises:

Test ticket (use this across all exercises)

From: sarah.chen@globalcorp.com Subject: Cannot access SSO login — entire team locked out Our team of 40 has been unable to log in via SSO since 09:00 this morning. We have a client demo in 3 hours. This is completely blocking us.

Your loop.py must include the following logic, in this order:

- Import the Anthropic SDK and your classify_ticket function. Initialise an Anthropic client.
- Define a tools list that registers classify_ticket as a callable tool for Claude. The JSON schema must declare ticket_text (string) and fields_needed (array of strings) as required properties, with a clear description for each.
- Build the initial messages list with one user-role entry. The prompt should instruct Claude to classify the ticket fully — all three fields — using the tool as many times as needed until all are confirmed.
- Enter a while True loop. On each iteration, call client.messages.create() passing the model name, max_tokens, your tools list, and the current messages. Print the iteration number and stop_reason so you can watch the loop execute.
- Immediately after the API call — before any branching — append the assistant response to messages. This step is mandatory and must come first.
- If stop_reason is "end_turn": print the final text content from the response, then break.
- If stop_reason is "tool_use": loop over response.content to find tool_use blocks. For each one, call the corresponding Python function using block.input as keyword arguments. Collect each result into a tool_result message (with the correct tool_use_id). Append all results as a single user turn, then continue the loop.

Step 3 — Run and observe

Run your file and verify the following before moving on:

```
python3 loop.py
```

- stop_reason is printed on every iteration.
- Tool calls are visible between loop iterations.
- The loop terminates only when all three classification fields appear in the final output.

Reflection Questions

- How many tool calls does Claude make? Is this consistent across runs? Why might it vary?
- What happens if you append tool results before appending the assistant turn? Try it and note the error.
- Replace while True with for i in range(2). What breaks, and what does it tell you about using iteration counts as a loop-exit strategy?

Exercise 2: Coordinator & Subagents (S2) ~15 min

Background

The classifier from Exercise 1 is one specialised agent. In a real triage pipeline you need four: Classifier, CRM Enricher, Drafter, and Validator. A coordinator receives the ticket, decides what needs to happen, and calls each subagent in turn.

The critical insight for this exercise is that subagents share no memory. A subagent knows only what the coordinator passes to it in that call's messages array. If you do not pass the classification result to the Drafter, the Drafter will guess — or hallucinate — the product area and severity.

Which model should I use for each agent?

Match model strength to the task — this keeps cost and latency in check without sacrificing quality:

- **Coordinator (orchestrator):** `claude-opus-4-6` — highest reasoning capability; it decides routing, owns state, and makes the critical judgement calls.
- **Each subagent (Classifier, Enricher, Drafter, Validator):** `claude-haiku-4-5-20251001` — fast and cost-efficient for focused, single-responsibility tasks. In production, each subagent can be switched to a different model independently without touching the coordinator. (The lab guide refers to this model as `claude-haiku-4-5` for brevity; the full API string used in code is `claude-haiku-4-5-20251001` — they refer to the same model.)

Note on coordinator model usage: In these exercises the coordinator (`coordinator.py`, `coordinator_v2.py`, `coordinator_v3.py`) is implemented as plain Python orchestration — it delegates every Claude API call to the subagent functions and does not call the API itself. The `claude-opus-4-6` model spec above applies when the coordinator makes its own Claude API calls to decide routing dynamically — a pattern used in more advanced multi-agent systems beyond this lab.

Task

Build a coordinator that calls four subagents in order and prints each output. Keep each subagent call as a separate function to make the isolation visible.

Step 1 — Create subagents.py

Write four functions in subagents.py. Each function makes exactly one client.messages.create() call and returns its result. The function signatures, system prompts, and expected return types are:

Function	Arguments	System prompt instruction	Returns
`run_classifier()`	ticket: str	Classify into product_area, severity, intent. Respond only in JSON.	**dict (parsed JSON)**
`run_crm_enricher()`	customer_email: str, classification: dict	Simulate a CRM lookup. Return account_tier, sla_tier, account_manager.	**dict**
`run_drafter()`	ticket: str, classification: dict, crm: dict	Draft a professional first-response email referencing the SLA tier.	**str**
`run_validator()`	draft: str, classification: dict, crm: dict	Check product area, SLA match, and tone. Reply APPROVED or list issues.	**str**

Important implementation notes for subagents.py:

- run_classifier must parse the JSON response defensively — strip any markdown code fences (``` or ```json) before calling json.loads().

- run_crm_enricher can return a hardcoded dict for this exercise. In production it would call a CRM API via an MCP tool. Include at least: account_tier, sla_tier, account_manager, contract_value.
- run_drafter should compose a context string from all three arguments and pass it as the user message content.
- run_validator should include the SLA tier and account tier in its user message so it has a concrete contract to check against.

Step 2 — Create coordinator.py

Write coordinator.py that imports all four subagent functions and calls them in sequence: Classifier → CRM Enricher → Drafter → Validator. After each call, print a labelled line showing the subagent name and its output. Pass the output of each step as input to the next step that needs it.

Run your coordinator with:

```
python3 coordinator.py
```

Memory Isolation Experiment

Modify run_drafter so it only receives the raw ticket string — remove the classification dict and crm dict from its input. Run coordinator.py again. Compare the output to the previous run. Note specifically: does the draft still reference the correct product area and SLA tier? This is the failure mode that explicit context passing prevents.

Restore run_drafter to its original signature before moving on.

Reflection Questions

- Could you pass the entire messages list from the Exercise 1 loop to each subagent instead of structured results? What are the token cost and accuracy implications?
- The Validator never uses tools. What stop_reason will it always return, and how does that simplify its caller code in the coordinator?

Exercise 3: Explicit Context Passing (S3) ~10 min

Background

Passing raw variables between functions works in small scripts. In a production pipeline it becomes fragile: you do not know what has been populated, what is still pending, and what was intentionally left blank versus accidentally omitted. A typed context object solves all three: it is self-documenting, enforces required fields at construction time, and makes partial-completion state visible.

Task

Introduce a TicketContext dataclass that carries all state through the pipeline. Verify that attempting to construct it with missing required fields fails loudly at the Python level — not silently in a Claude response.

Step 1 — Create context.py

Define a Python dataclass called TicketContext using the @dataclass decorator. Structure its fields in four groups, in this order:

- Required at intake (no default): ticket_id (str), raw_ticket (str), customer_email (str). These must be provided at construction — no Optional, no default value.
- Populated by Classifier (default None): product_area, severity, intent — all Optional[str].
- Populated by CRM Enricher (default None): account_tier, sla_tier, account_manager — all Optional[str].
- Populated by Drafter and Validator (default None): draft_response, validation_result — both Optional[str].

Add three helper methods to the class:

- classification_complete(self) -> bool: returns True only if product_area, severity, and intent are all non-None.
- enrichment_complete(self) -> bool: returns True only if account_tier and sla_tier are both non-None.
- draft_complete(self) -> bool: returns True only if draft_response is not None.

Step 2 — Create coordinator_v2.py

Copy coordinator.py to coordinator_v2.py and refactor it to use TicketContext:

- Construct a TicketContext at the top of the file with ticket_id, raw_ticket, and customer_email.
- After each subagent call, write its results back into the matching fields on ctx (e.g. ctx.product_area = result['product_area']).
- Pass only the specific ctx fields each subagent needs — not the entire ctx object.
- At the end, print the full ctx object to confirm all fields are populated.

```
python3 coordinator_v2.py
```

Reflection Questions

- The dataclass TypeError is raised at construction time. When would a missing key in a plain dict be discovered? Which failure mode is safer in a pipeline that runs unattended overnight?
- The helper methods (classification_complete, enrichment_complete, draft_complete) return bool. How will you use these in Exercise 4 to decide whether to allow the next step to proceed?

Exercise 4: Programmatic Step Enforcement (S4) ~10 min

Background

You could write the coordinator prompt as: "Classify first, then enrich, then draft, then validate — never skip a step." This works most of the time. It fails under token pressure, model drift, ambiguous inputs, or when a tool returns an error that the prompt did not anticipate. Programmatic gating means your Python code checks the precondition before making the next API call. If the gate fails, the pipeline stops with a clear, named error — not a subtly wrong output.

> **Prompt rule vs. gate**
> A prompt rule says: 'Don't draft until you have classified.' A gate says: if not ctx.classification_complete(): raise PipelineGateError(...). The first is advice the model can ignore under pressure. The second is a hard stop enforced by the runtime. Use gates for ordering in any pipeline where a wrong-order execution produces incorrect data rather than an obvious crash.

Task

Add gates between every step in the coordinator pipeline. Demonstrate that the gates block execution and produce informative errors — not silent failures.

Step 1 — Create gates.py

Define a custom exception class called PipelineGateError that extends Exception. Then write three gate functions:

- gate_classification(ctx): raises PipelineGateError if ctx.classification_complete() returns False. The error message must name which specific fields are missing (not just 'classification incomplete').
- gate_enrichment(ctx): raises PipelineGateError if ctx.enrichment_complete() returns False. The error message must state which enrichment fields are None and instruct the caller to rerun the CRM Enricher.
- gate_draft(ctx): raises PipelineGateError if ctx.draft_complete() returns False. The error message must confirm that draft_response is None and instruct the caller to rerun the Drafter.

Each gate function should return None on success (the pipeline continues) and raise PipelineGateError on failure (the pipeline stops).

Step 2 — Create coordinator_v3.py

Copy coordinator_v2.py to coordinator_v3.py. Add the following structure:

- Import PipelineGateError and all three gate functions from gates.py.
- Wrap the entire pipeline in a try / except PipelineGateError block.
- After Step 1 (Classify) and before Step 2 (Enrich): call gate_classification(ctx). Print a 'Gate 1 passed' confirmation if it succeeds.
- After Step 2 (Enrich) and before Step 3 (Draft): call gate_enrichment(ctx). Print a 'Gate 2 passed' confirmation if it succeeds.
- After Step 3 (Draft) and before Step 4 (Validate): call gate_draft(ctx). Print a 'Gate 3 passed' confirmation if it succeeds.
- In the except block: print the full exception message with a clear [PIPELINE BLOCKED] label.

```
python3 coordinator_v3.py
```

Step 3 — Prove the gate blocks

Deliberately trigger Gate 1 to confirm it behaves correctly. Create a new file called `coordinator_v3_sabotage.py` — a copy of `coordinator_v3.py` — and add the line `ctx.severity = None` immediately after the Classifier writes its results into ctx and before Gate 1 runs. Run `python3 coordinator_v3_sabotage.py` and observe that:

- PipelineGateError is raised immediately at Gate 1 — not somewhere later in the pipeline.
- The error message names severity as the missing field.
- Steps 2, 3, and 4 never execute.

The original `coordinator_v3.py` is unchanged and clean. Run `python3 coordinator_v3.py` to confirm the full pipeline runs cleanly before moving on. The solution notebook provides `coordinator_v3_sabotage.py` as a ready-to-run demo of this step.

Reflection Questions

- PipelineGateError names the missing fields in its message. Why is a named exception with context better than a bare assert statement?
- In a production system, should a gate failure automatically retry the failed step, or immediately alert a human? What factors would influence that decision?
- Gate 2 currently checks both account_tier and sla_tier. What if the CRM returns partial data — account_tier present but sla_tier None? Should the gate pass or fail? Modify gate_enrichment to handle this case explicitly.

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking at your code. If you cannot answer confidently, flag it for Doubt Session 2.

#	Question
1	Name all four stop_reason values. For each one, write one sentence describing what your loop code does when it encounters it.
2	You have a Drafter subagent. What exact data do you pass to it — and why not the entire conversation history from the coordinator?
3	Explain why the TypeError from a missing TicketContext field is a better failure than a Claude hallucination caused by missing context.
4	Your team lead says: 'Just put DO NOT DRAFT BEFORE CLASSIFYING in the system prompt — that's the same as a gate.' How do you respond?
5	Without looking at the code: after two tool calls and their results, how many messages are in the messages array? What are their roles?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Forgetting to append the assistant turn before appending tool results.	The messages array becomes malformed. Claude will error or behave erratically. Always append assistant content first, tool results second.
Exiting the loop when tool_results is empty rather than checking stop_reason.	Claude may call tools zero times (it might reason without calling) or many times. stop_reason is the only reliable signal.
Passing the full coordinator message history to every subagent.	Wastes tokens, leaks irrelevant context, and may confuse the subagent. Pass only the fields that subagent needs.
Using a prompt instruction instead of a gate for step ordering.	Works most of the time, silently fails under edge cases. Gates give you a named error and a clear recovery path.
Assuming the classifier JSON is always well-formed.	LLMs occasionally include markdown fences or trailing commas. Always strip and parse defensively; gate on parse failure.

6. Quick Reference

6.1 Agentic Loop — Skeleton

```
messages = [{'role': 'user', 'content': initial_prompt}]

while True:
    response = client.messages.create(model=..., tools=..., messages=messages)
    messages.append({'role': 'assistant', 'content': response.content})  # always first

    if response.stop_reason == 'end_turn':
        # extract text and break
        break

    if response.stop_reason == 'tool_use':
        tool_results = []
        for block in response.content:
            if block.type == 'tool_use':
                result = dispatch_tool(block.name, block.input)

tool_results.append({'type':'tool_result','tool_use_id':block.id,'content':result})
        messages.append({'role': 'user', 'content': tool_results})
```

6.2 Coordinator Pattern — Skeleton

```
# Coordinator owns the TicketContext and calls subagents explicitly.
# Each subagent is a separate client.messages.create() call — no shared state.

ctx = TicketContext(ticket_id=..., raw_ticket=..., customer_email=...)

# Subagents receive only what they need — never the full context object.
classification = run_classifier(ctx.raw_ticket)
crm_data       = run_crm_enricher(ctx.customer_email, classification)
draft          = run_drafter(ctx.raw_ticket, classification, crm_data)
verdict        = run_validator(draft, classification, crm_data)
```

6.3 File Inventory

File	Exercise	Purpose
tools.py	Ex 1	Simulated classification tool callable from the agentic loop.
loop.py	Ex 1	Agentic loop with correct stop_reason handling and tool dispatch.
subagents.py	Ex 2	Four subagent functions: Classifier, CRM Enricher, Drafter, Validator.
coordinator.py	Ex 2	First coordinator — calls subagents in sequence, no context object.
context.py	Ex 3	TicketContext dataclass with completion-check helper methods.
coordinator_v2.py	Ex 3	Refactored coordinator using TicketContext for all state.
gates.py	Ex 4	PipelineGateError and three gate functions.
coordinator_v3.py	Ex 4	Final coordinator with all gates wired between steps.

File	Exercise	Purpose
coordinator_v3_sabotage.py	Ex 4	Gate-block demo (Ex 4 Step 3): identical to coordinator_v3.py but with ctx.severity = None added after the Classifier to deliberately trigger Gate 1.

6.4 Further Reading

- Anthropic API — Tool use & agentic loops: docs.anthropic.com/en/docs/agents
- Claude Code 101 on Skilljar — Session management (S7)
- Introduction to Subagents on Skilljar — Parallel Task calls (S3)
- Building with the Claude API on Skilljar — Programmatic gating (S4)